

PIN10 : Projet 3A

Programmation Orientée Objets en Java

Sujet original de Stéphanie Chollet avec ajouts de Jean-Luc Koning

Mai-Juin 2026

Table des matières

Remarques préliminaires.....	3
À l'attention des utilisateurs Windows.....	3
Sauvegarde de votre travail.....	4
Modalités d'évaluation.....	4
Configuration de l'environnement de développement Eclipse.....	4
Introduction : à propos de l'IDE Eclipse.....	5
Mise à jour des variables d'environnement.....	5
Configuration du « Build Path » d'Eclipse.....	5
Étape 1.....	6
Étape 2.....	11
Partie 1 : Compilation et exécution du programme <i>HelloWorld</i> avec Apache/Ant (les 2 ou 3 premiers créneaux horaires).....	12
Initialisation.....	12
Compilation et exécution du code.....	13
Passage d'arguments.....	14
Analyse de votre travail et éléments à retenir.....	15
Utilisation de Ant pour compiler et exécuter votre code.....	16
Création package fr.esisar.helloworld.....	16

Ligne de commande depuis un terminal.....	16
Fichier build.xml.....	17
Documentation Javadoc.....	18
Clean.....	19
Version de build.xml avec variables génériques.....	19
Partie 2 : Objets géométriques (les 6 ou 7 derniers créneaux horaires).....	19
Avant de commencer.....	19
Première classe.....	19
Projet Eclipse.....	19
Package fr.esisar.geometrie.....	19
Classe Point.....	19
Classe GeometrieTest.....	20
Questions :.....	21
Ajout de méthodes à la classe Point.....	21
Diagramme d'objets.....	24
Code de la méthode main().....	24
Questions.....	24
Classe Cercle.....	25
Package fr.esisar.geometrie.....	25
Code de la méthode main().....	25
Ajout de deux méthodes.....	26
Questions.....	26
Formater l'affichage des chaînes de caractères.....	26
Produire un rendu graphique.....	28

Classe abstraite et interface.....	29
Classe abstraite ObjetGeometrique.....	30
Révision du code de la classe Cercle.....	30
Classe de test TestObjetGeometrique.....	30
Classe CercleComparable.....	30
Classe CerclesTriés.....	30
Gestion des exceptions.....	31
Classe CercleAvecException.....	31
Classe TestCercleAvecException.....	31
Développement d'une interface graphique avec JavaFX.....	31
Installation de JavaFX.....	31
Classe AfficheCercle.....	32
Interaction graphique avec les cercles.....	33
Création interactive des anneaux olympiques.....	33

Remarques préliminaires

L'utilisation de votre machine individuelle est autorisée.

Dans ce cas, pensez à bien adapter les consignes de l'énoncé. Notamment, configurez votre environnement pour utiliser la dernière version du JDK installé sur votre machine.

À l'attention des utilisateurs Windows

Vous devez avoir à tout moment au moins un terminal ouvert pour pouvoir entrer des commandes Unix (shell).

Vous pouvez utiliser "PowerShell" depuis un terminal Windows. Mais il ne s'agit pas d'un véritable shell avec toutes les commandes Unix.

Il vous est recommandé d'installer une distribution Linux sur votre poste en utilisant WSL2 (Windows Subsystem for Linux)

Étapes à suivre :

1. Ouvrir un terminal en mode administrateur
2. Vérifier si WSL Ubuntu 22.04 est déjà installée : `wsl -l`
3. Installer WSL avec la commande : `wsl --install -d Ubuntu-22.04`
4. Mettre à jour les différents packages Linux nécessaires.

N'oubliez pas d'installer tous les outils nécessaires à ce projet. Exemple : Ant.

Sauvegarde de votre travail

Pour ceux d'entre vous qui font le choix de travailler sur les machines de l'école, pensez bien à sauvegarder votre travail à la fin de chaque créneau.

En effet, les machines des salles (B141 notamment) sont réinitialisées deux fois par jour : la nuit et entre midi et deux heures.

Pour cela, vous pouvez exporter votre projet Eclipse (et non le workspace) :

- clic droit sur le projet
- Export > General > Archive file
- tout garder au format zip ou tar.
- Déposer cette archive soit sur une clé usb soit sur votre cloud personnel, ou encore sur le cloud de l'école (exemple : *Briefcase Zimbra*).

Modalités d'évaluation

- Ce travail est individuel
- les deux dernières séances seront consacrées à une évaluation individuelle orale sur la base du code produit et de votre compréhensions des différentes notions traitées dans ce sujet.

Configuration de l'environnement de développement Eclipse

Compétences visées :

- Maîtrise de la configuration de son environnement de travail (système d'exploitation, versions de Java, Eclipse).

- Maîtrise du processus de développement logiciel d'une application développée en langage Java.
- Apprentissage de l'outil d'automatisation des tâches Apache Ant.
- Apprentissage de la documentation de code avec la Javadoc.

Introduction : à propos de l'IDE Eclipse

Eclipse Brief Tutorial : https://mediaplayer.pearsoncmg.com/assets/secs-liang-Eclipse_Brief_Tutorial

Si vous ne l'avez pas déjà fait pour les TP du cours de Java IN360, visionnez le tutorial Eclipse et testez les fonctionnalités présentées dans la vidéo avant de vous lancer dans le projet.

Instructions de démarrage avec exemples pour se lancer dans l'utilisation d'Eclipse : https://media.pearsoncmg.com/ph/esm/ecs_liang_java_13e/cw/content/Supplement2fEclipseStartup.docx

Mise à jour des variables d'environnement

Vérifiez la configuration de votre environnement de travail. Quelles sont les versions du JDK et du JRE de votre machine ?

Si les versions ne sont pas les mêmes, *modifiez vos variables d'environnement* afin d'avoir une configuration correcte.¹

Configuration du « Build Path » d'Eclipse

Dans Eclipse², créez un projet Java nommé HelloWorld_v1.

Ce projet ne comprend pas de modules.

Vous devez configurer le « /Build Path/ » de votre projet afin qu'Eclipse utilise le même JDK/JRE que celui installé par défaut sur votre machine.

La configuration se fait en 2 étapes :

¹ Par défaut, l'ensemble des machines de l'école sont configurées correctement pour l'utilisation de Java sous Linux.

² Par défaut, Eclipse est installé sur l'ensemble des machines de l'école sous Linux. Eclipse est disponible, pour certaines salles de l'école, uniquement en ligne de commandes depuis une console. Démarrez de préférence Eclipse en tâche de fond avec la commande : `eclipse &`

1. la première consiste à indiquer où se trouve le JDK à utiliser,
2. la deuxième permet d'indiquer à Eclipse dans quelle version de Java vous travaillez notamment pour qu'il compile et exécute votre code.

Cette configuration (étapes 1 et 2) DOIT ÊTRE FAITE POUR CHAQUE nouveau projet dans Eclipse.

Étape 1

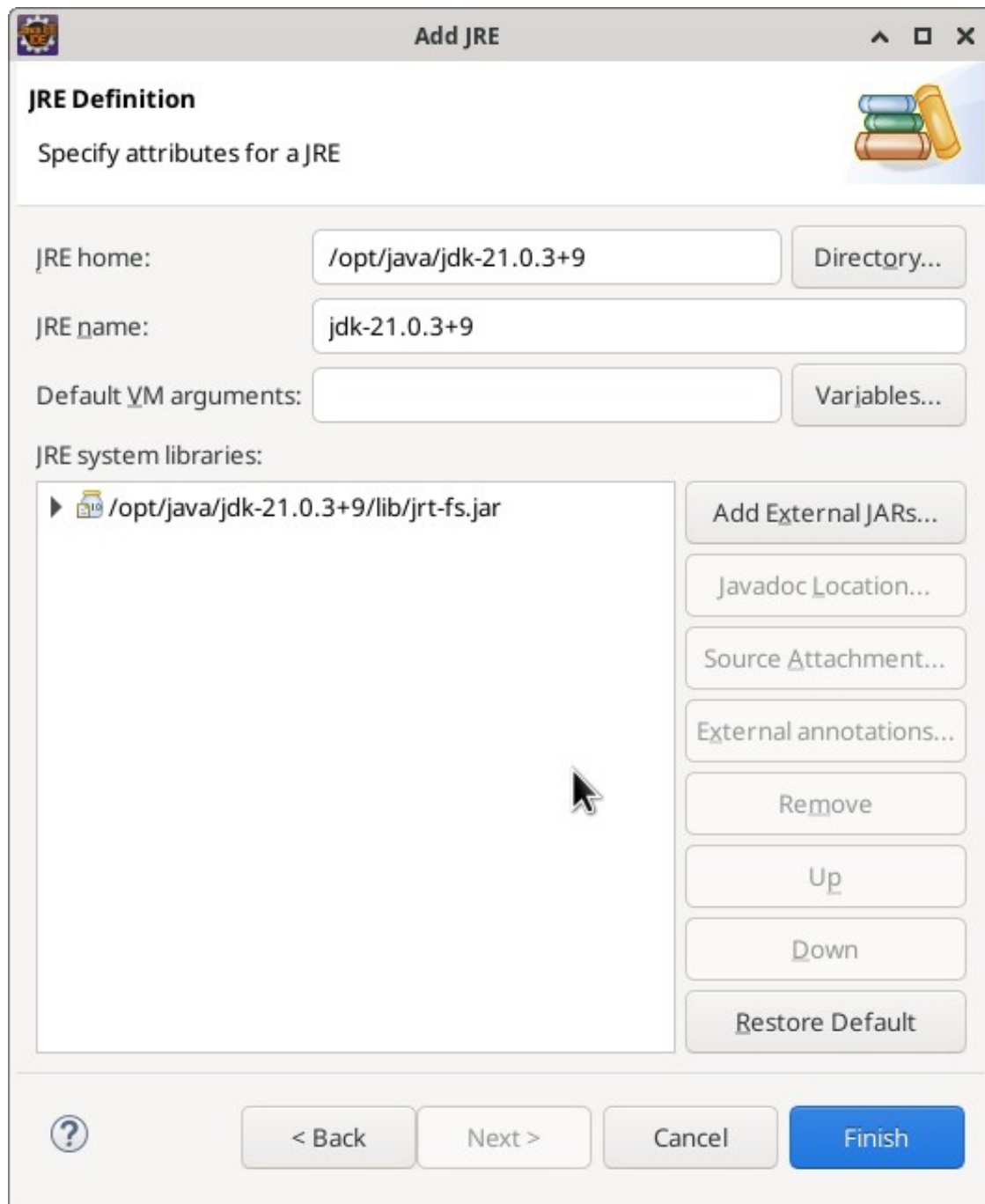
Pour réaliser la configuration du « /Build Path/ » d'Eclipse :

1. Clic droit sur le projet puis Build Path > Configure Build Path...
2. Dans l'onglet Libraries, supprimer le JRE actuel.
3. Sélectionner Modulepath, puis Add Library.
4. Dans la nouvelle fenêtre choisir, JRE System Library, Next,
5. Cliquer sur le bouton Installed JREs,
6. Supprimer la version présente,
7. Ajouter avec le bouton Add.
8. Sélectionner Standard VM, Next,
9. Cliquer sur le bouton Directory et choisir l'emplacement de votre JDK (Figure 2).

Sur les machines de l'école, Java est installé dans le répertoire
`/opt/java/jdk-21.0.4+7`.

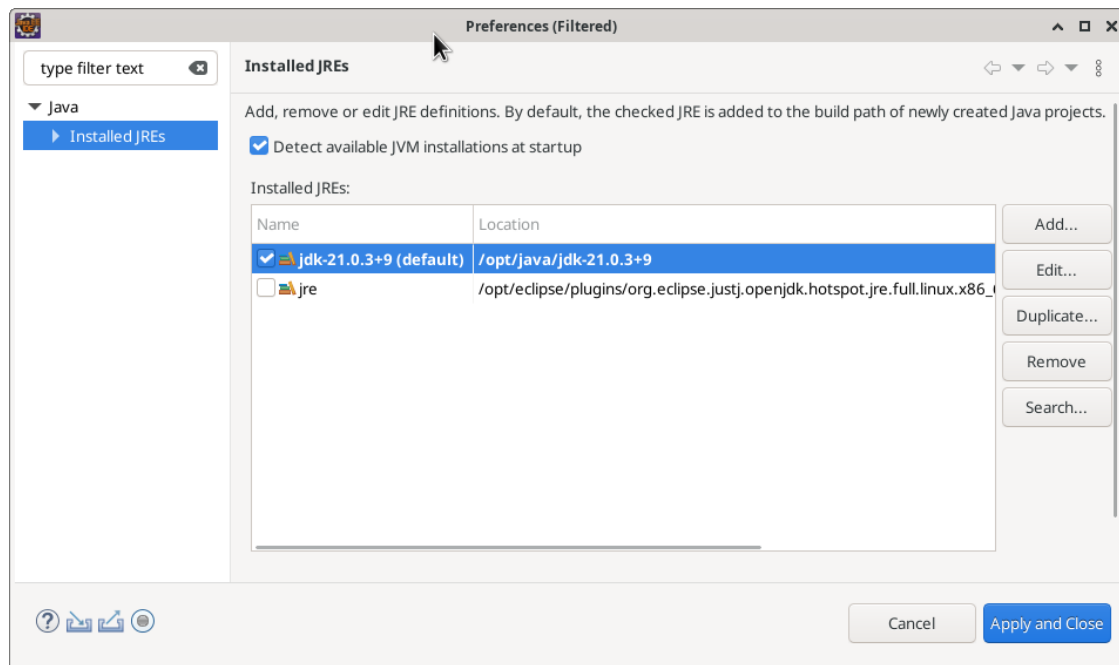
Si vous utilisez votre propre machine, faites pointer vers la dernière version de Java installée. Recherchez préalablement sa localisation.

10. Cliquer sur Finish



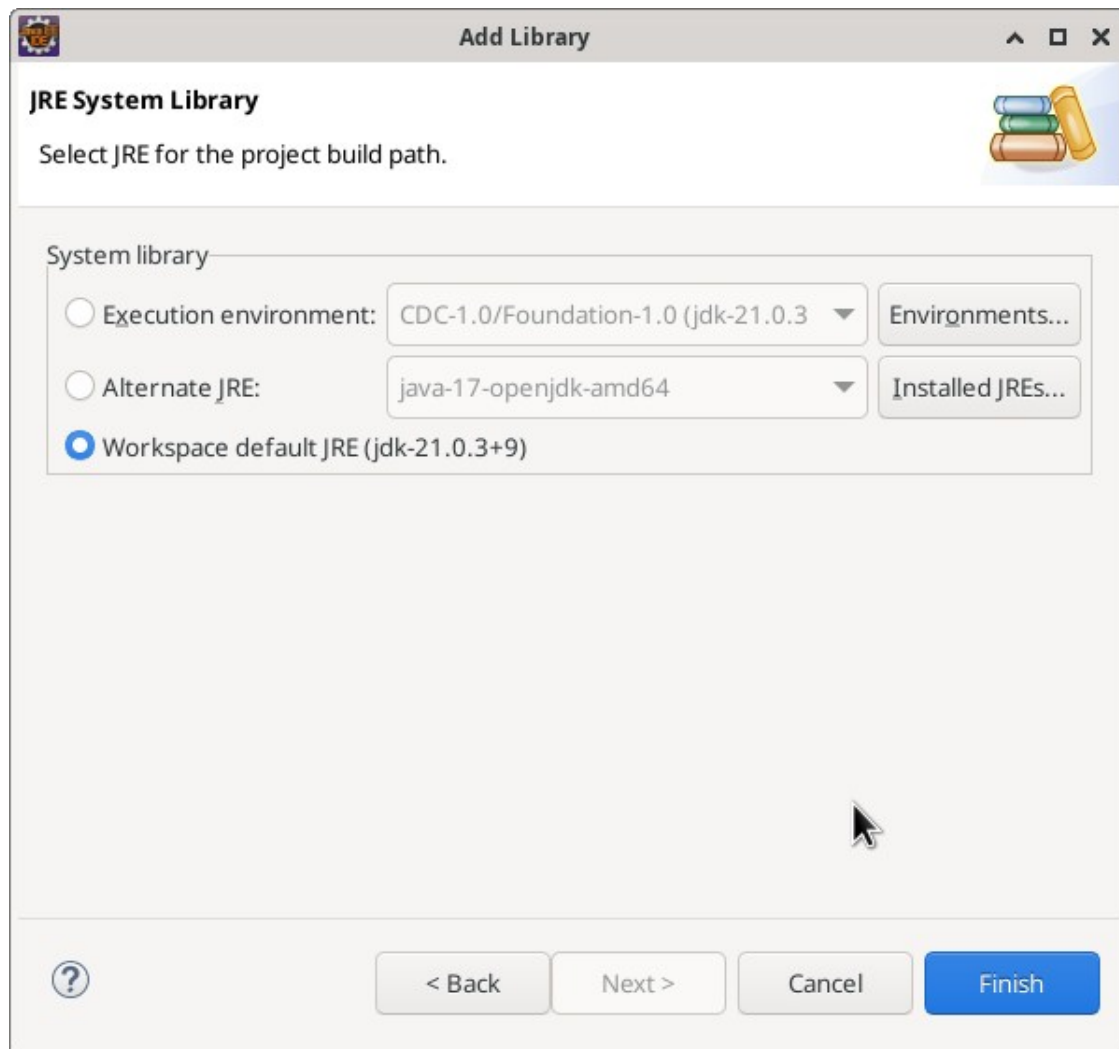
Définition du JDK/JRE

11. Sélectionnez le bon JDK (c'est-à-dire le nouveau que vous avez ajouté) dans la fenêtre « Installed JRE » (Figure 3), puis cliquez sur Apply and Close.



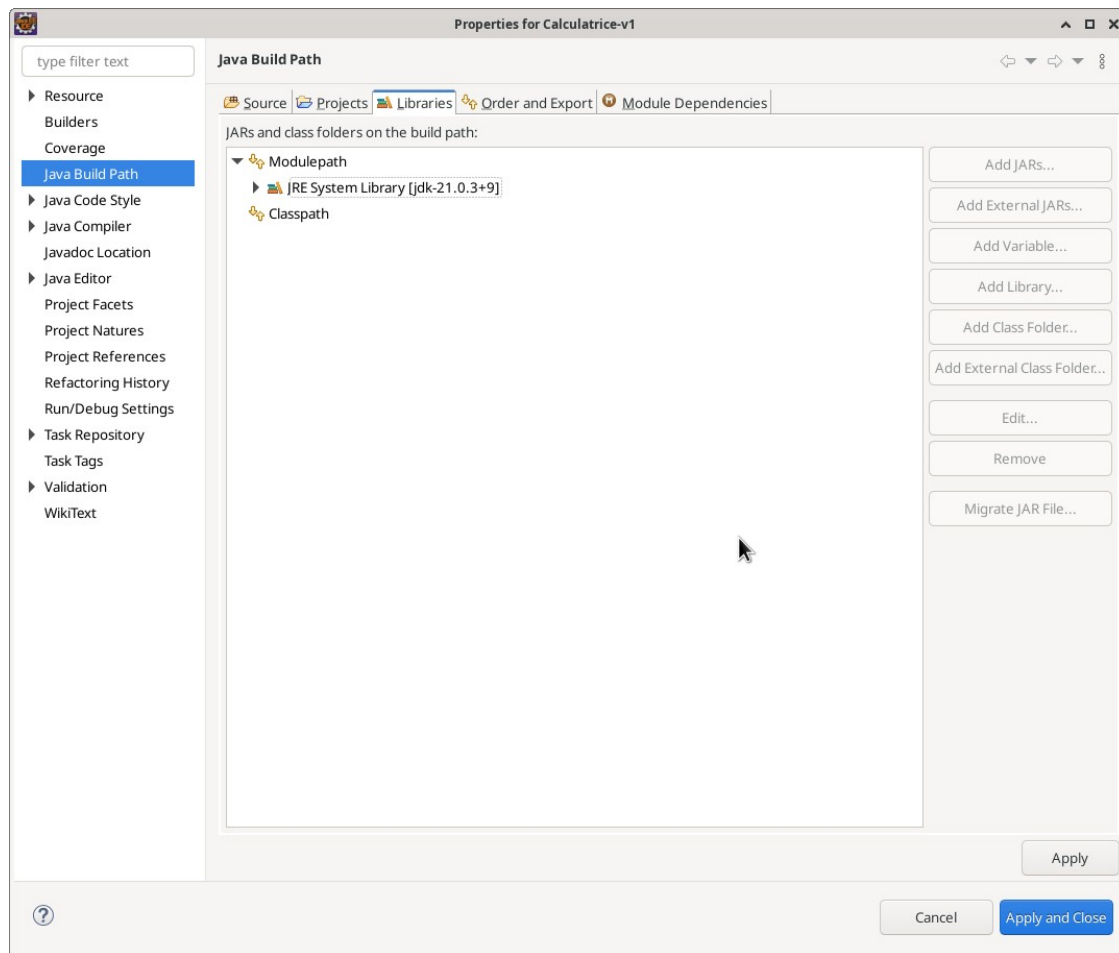
Installed JREs

12. Enfin, vous devez obtenir que le *Workspace default JRE* (Figure 4) utilise la version que vous avez choisie.



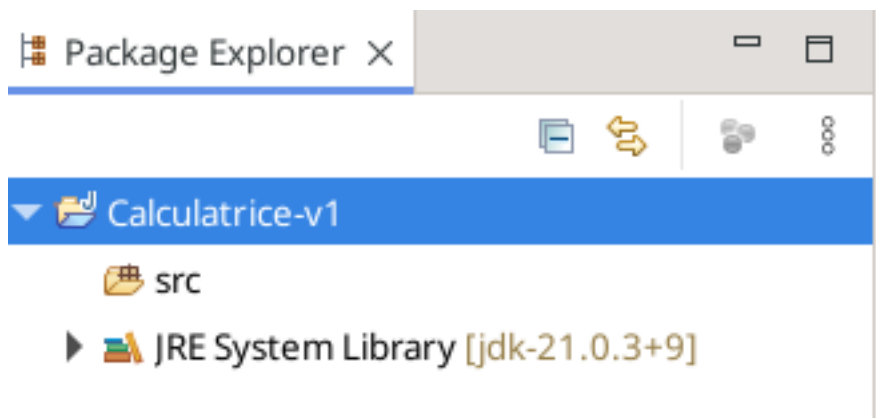
JRE System Library

13. Cliquer sur Apply and Close dans la fenêtre *Java Build Path* (Figure 5).



Java Build Path

Pour vérifier que votre projet utilise le bon JDK, vous devez le voir apparaître dans l'arborescence de votre projet dans la fenêtre *Package Explorer* (Figure 6).



Arborescence du projet

Étape 2

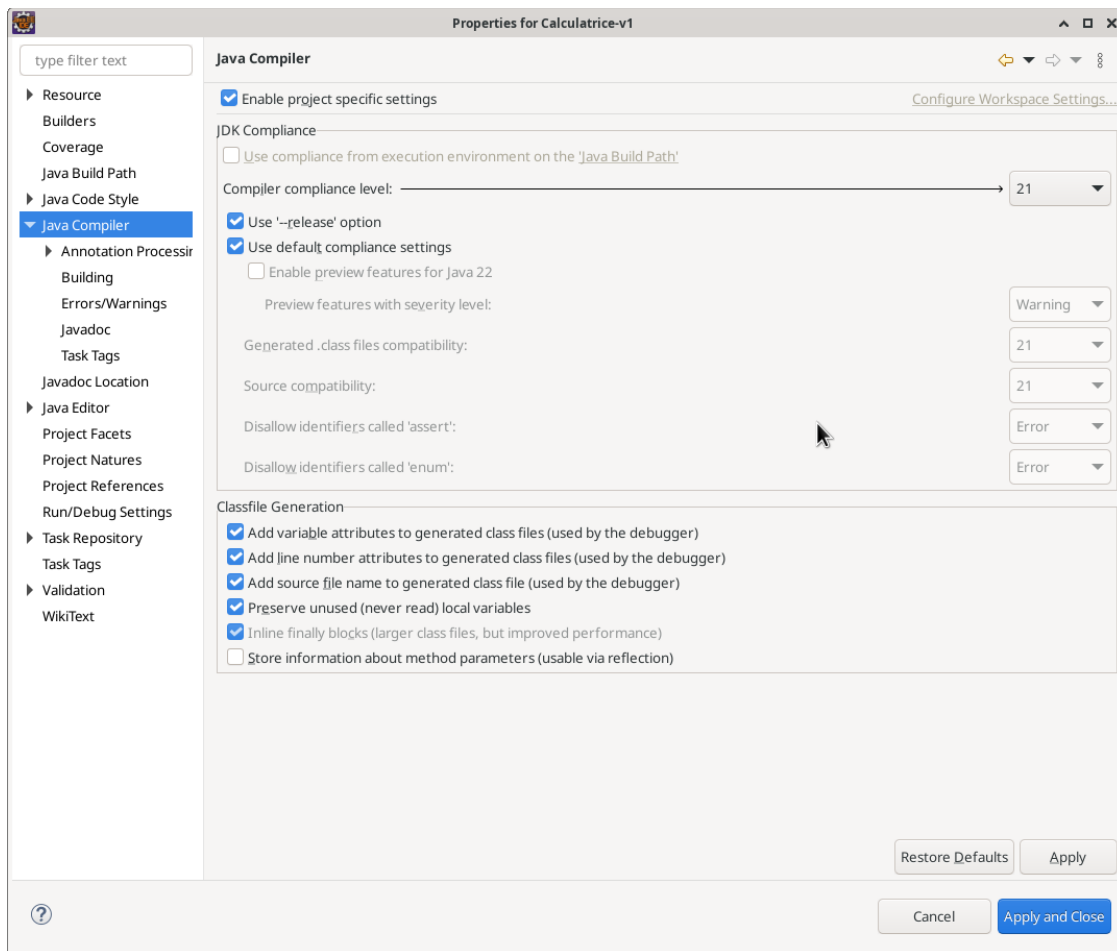
1. Vous pouvez vérifier dans une console de commande les versions qui seront réellement utilisées pour le compilateur et l'interpréteur java de la façon suivante.

```
$ javac --version
```

```
$ java --version
```

2. Clic droit sur votre projet, Properties > Java Compiler (Figure 7).

Il faut préciser pour *Compiler compliance level* que vous travaillez en version 21, puis Apply and Close.



Partie 1 : Compilation et exécution du programme *HelloWorld* avec Apache/Ant (les 2 ou 3 premiers créneaux horaires)

Un rappel des principales instructions java est disponible dans le fichier *CheatSheet-Java.pdf* déposé dans l'espace Chamilo.

<https://chamilo.grenoble-inp.fr/main/document/document.php>

Initialisation

1. Créez un projet nommé HelloWorld_v1, qui ne comprend pas de modules. (Cf. instructions ci-dessus.)

Votre projet doit contenir à présent un répertoire `src/` dans lequel vous allez mettre l'ensemble de vos fichiers `.java` contenant le code que vous allez développer. Et, vous trouvez aussi la version de Java que vous utilisez avec l'élément « `/JRE System Library/` » avec entre crochets ("`[jdk-21.0.4+7]`") la version utilisée.

En dépliant cet élément *JRE System Library*, que retrouvez-vous ?

2. Dans le répertoire `src/`, créez un package nommé `fr.esisar.helloworld`
3. Dans le package `fr.esisar.helloworld`, créez une classe nommée *HelloWorld* contenant la méthode `main()`.

Le code suivant est automatiquement généré :

```
package fr.esisar.hello;

public class HelloWorld {
    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }
}
```

La ligne 5 contient un commentaire automatiquement généré. Vous devez la supprimer sitôt que vous avez complété le code de la méthode concernée.

Complétez le code de la méthode `main()` pour afficher le message « `/Hello World !/` » avec la commande `System.out.println()`.

Compilation et exécution du code

1. Eclipse est un environnement de développement intégré qui facilite les différentes opérations faites par le développeur, notamment, l'écriture du code, la compilation et l'exécution d'applications. Il est donc possible d'exécuter votre programme depuis Eclipse : clic droit sur la classe `HelloWorld.java`, puis `Run As > Java Application`.

Il existe aussi un raccourci dans le bandeau en d'actions : la flèche verte



. Le résultat de l'exécution est affiché dans la console d'Eclipse.

Cependant, une application Java n'est jamais fournie au client avec le logiciel Eclipse. Votre code doit être compilé et exécuté à l'extérieur d'Eclipse, soit en ligne de commandes sur votre machine. De plus, vous devez fournir au client le code exécutable et non le code source.

2. Par conséquent, vous devez créer un répertoire `classes/` dans le répertoire de votre projet `workspace/HelloWorld_v1`. Votre projet sera donc constitué des éléments suivants :
 - un répertoire `.settings` ainsi que deux fichiers `.classpath` et `.project`, qui contiennent les configurations Eclipse spécifiques à votre projet ;
 - un répertoire `bin/` utilisé par Eclipse pour compiler et exécuter votre projet dans Eclipse,
 - le répertoire `src/` dans lequel vous devez retrouver votre code avec un répertoire par élément du package : `fr/esisar/helloworld/` puis le fichier `HelloWorld.java`,
 - et enfin le répertoire `classes/` que vous avez créé et dans lequel vous allez mettre la classe *HelloWorld* compilée.

```
workspace/HelloWorld_v1$ javac src/fr/esisar/helloworld/HelloWorld.java  
-d classes
```

Vérifiez que le fichier `HelloWorld.class` a bien été généré. Est-ce que l'arborescence a été conservée ?

Vous pouvez afficher dans Eclipse l'arborescence actuelle de votre projet en le rafraîchissant. Pour cela,

- cliquez sur le projet, puis F5, ou bien
 - clic droit sur le projet puis Refresh.
3. Vous pouvez à présent exécuter en ligne de commandes votre code compilé :

```
workspace/HelloWorld_v1$ java -cp classes fr.esisar.helloworld.HelloWorld
```

A quoi sert l'option -cp ? Si vous retirez l'option « -cp classes », quel message d'erreurs obtenez-vous ?

Est-ce qu'il est possible d'exécuter votre programme avec cette ligne de commandes :

```
workspace/HelloWorld_v1$ java -cp classes HelloWorld
```

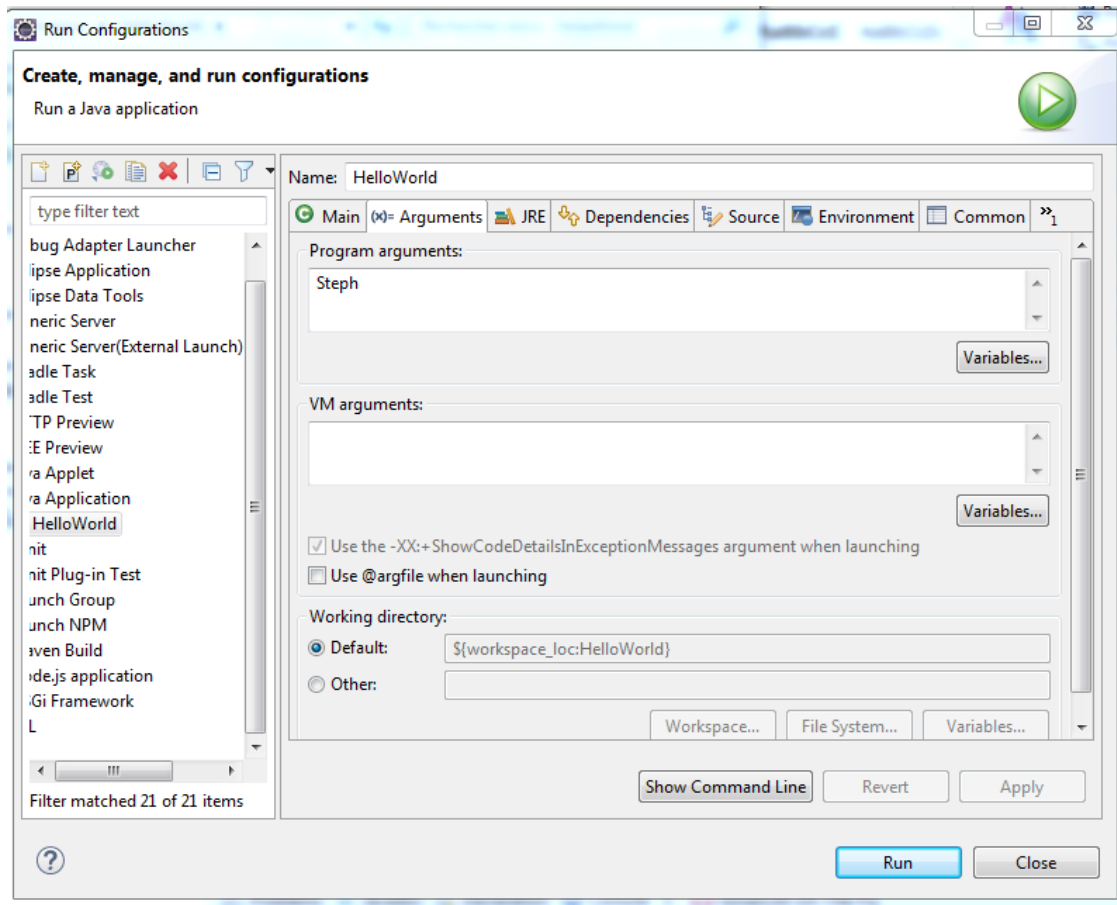
Quel est le message d'erreurs ?

Dans les deux cas, pourquoi n'est-il pas possible d'exécuter votre programme ?

Passage d'arguments

1. Modifiez votre programme pour dire bonjour à la personne que vous avez passée en paramètre. En l'absence de paramètre, votre application continuera de dire « Hello World! ».
2. Vérifiez que votre programme s'exécute correctement depuis Eclipse avec et sans argument.

Pour passer en paramètres des arguments lors de l'exécution de votre programme, clic droit sur la classe principale, Run As > Run Configurations... Puis dans l'onglet « Arguments », dans la partie "Program arguments", ajoutez la valeur que l'on souhaite passer :



Passage d'arguments

En exécutant le programme avec cette configuration, vous devez obtenir le message « Hello Steph! »

3. Compilez et exécutez votre programme en ligne de commandes. Pour passer l'argument en ligne de commandes, vous devez l'ajouter à la commande vue précédemment :

```
workspace/HelloWorld_v1$ java -cp classes fr.esisar.helloworld.HelloWorld Steph
```

Analyse de votre travail et éléments à retenir

Répondez aux questions suivantes sous forme de commentaires dans votre code.

1. Quelle est la première chose à vérifier si, lors de l'exécution de votre programme, vous obtenez le message suivant :

```
Exception in thread "main" java.lang.NoClassDefFoundError:
HelloWorld.java
```

2. Quelle est la « bonne » signature de la méthode `~main()` ?
3. Lorsque vous compilez un programme écrit dans le langage de programmation Java, le compilateur convertit le fichier source lisible par l'homme en code indépendant de la plate-forme qu'une machine virtuelle Java peut comprendre.

Comment s'appelle ce code indépendant de la plate-forme ?

Utilisation de Ant pour compiler et exécuter votre code

/En préparation du travail pour cette section, veuillez revoir le cours sur **Ant** donné en IN360 par Stéphanie Chollet dont les slides intitulées "Processus de développement logiciel Java et automatisation avec Apache Ant" sont accessibles dans l'espace Chamilo./

<https://chamilo.grenoble-inp.fr/main/document/document.php>

Création package `fr.esisar.helloworld`

Dans votre projet `HelloWorld_v1`, créez un package `fr.esisar.helloworld` qui comprendra l'ensemble des classes développées dans cet exercice.

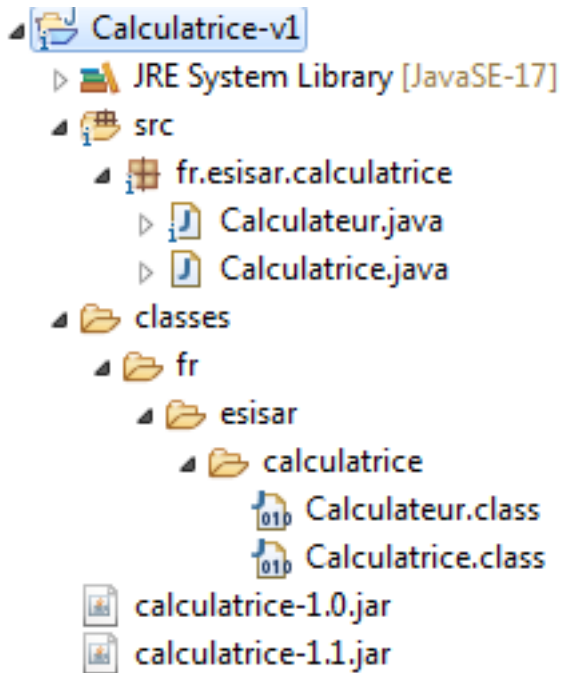
Ligne de commande depuis un terminal

En ligne de commande depuis un terminal, vous devez depuis la racine du projet, c'est-à-dire le répertoire `=HelloWorld_v1=` :

- **Compiler** votre code source en mettant le résultat de la compilation dans un répertoire nommé `classes` à la racine du projet. Le résultat de votre ligne de commande doit automatiquement mettre tous les fichiers compilés dans ce répertoire `classes` en gardant l'arborescence du répertoire `src`. Vous ne devez pas déplacer manuellement les fichiers compilés dans ce répertoire `classes` et vous ne devez pas créer manuellement l'arborescence du répertoire `classes`.
- **Exécuter** votre code depuis la racine de votre projet.
- **Packager** le code compilé dans une archive non exécutable nommée `helloworld-1.0.jar` à la racine de votre projet, puis exécuter cette archive depuis la racine de votre projet

- **Packager** votre code dans une archive exécutable nommée helloworld.jar à la racine de votre projet, puis exécuter cette archive depuis la racine de votre projet.

L'arborescence du projet après l'exécution des différentes commandes doit correspondre à celle donnée en exemple ci-après (Figure 9).



Exemple d'arborescence du projet après exécution des différentes commandes

Fichier build.xml

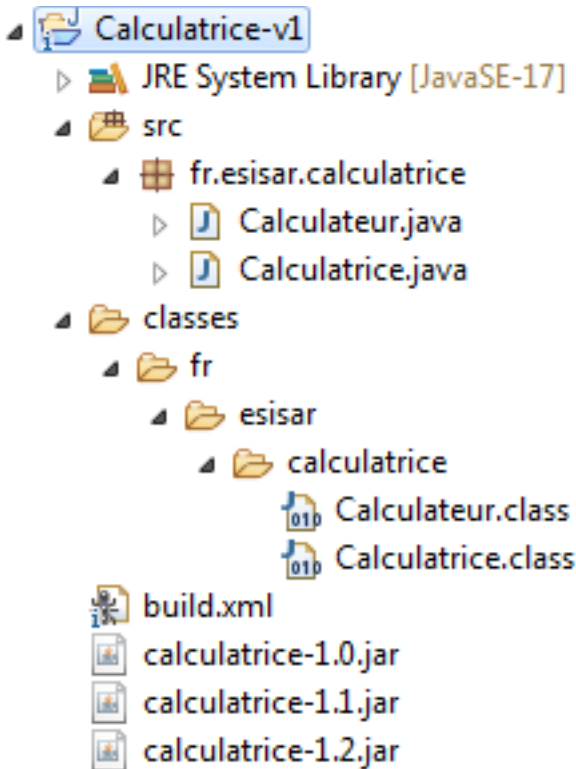
Créez un fichier build.xml à la racine de votre projet contenant les commandes qui permettent d'automatiser les tâches de compilation, de packaging et d'exécution de votre programme.

Votre fichier build.xml doit contenir les tâches suivantes :

- **compile** qui crée le répertoire classes et génère les classes compilées dans ce répertoire classes.
- **package** qui crée à la racine du projet une archive exécutable nommée helloworld-1.2.jar
- **run** qui exécute l'archive helloworld-1.2.jar

- **clean** qui supprime le répertoire classes ainsi que l'archive helloworld-1.2.jar.

L'arborescence de votre projet doit correspondre à celle donnée en exemple ci-après (Figure 10).



Exemple d'arborescence complète du projet

Documentation Javadoc

Ajoutez la documentation Javadoc à votre code pour la classe HelloWorld ainsi que pour l'ensemble de ses méthodes.

Pour vous aider : <https://fr.wikipedia.org/wiki/Javadoc>

1. Création de la documentation via Ant

Ajoutez une cible dans build.xml permettant de générer automatiquement la Javadoc dans un répertoire nommé doc se situant à la racine du projet. Vous devez utiliser la commande javadoc avec les attributs sourcepath et destdir.

Pour vous aider : <https://ant.apache.org/manual/Tasks/javadoc.html>

Clean

1. Ajoutez une tâche clean-javadoc qui supprimer le répertoire contenant la javadoc.
2. Ajoutez une tâche clean-all qui appelle les deux tâches de nettoyage clean et clean-javadoc pour nettoyer complètement votre projet.

Pour vous aider : <https://ant.apache.org/manual/Tasks/antcall.html>

Version de build.xml avec variables génériques

Si vous souhaitez approfondir votre utilisation de Ant, vous pouvez produire une version générique de votre fichier build.xml en faisant appel aux variables de type "\${xxx}".

Partie 2 : Objets géométriques (les 6 ou 7 derniers créneaux horaires)

Compétences visées :

- Manipulation de classes et instanciation d'objets.
- Utilisation des diagrammes de classes et d'objets.

Avant de commencer

1. Assurez-vous que votre environnement de travail est bien configuré !
2. Pour ce second projet, il est également demandé d'utiliser Ant. Il vous appartient d'adapter votre fichier build.xml.

Première classe

Projet Eclipse

Créez un projet Eclipse nommé Geometrie.

Package fr.esisar.geometrie

Créez un package fr.esisar.geometrie dans le répertoire src/

Classe Point

Créez la classe Point pour représenter des points dans un repère orthonormé.

La classe Point doit être implémentée pour correspondre au diagramme de classes¹ donné à la figure 11.²

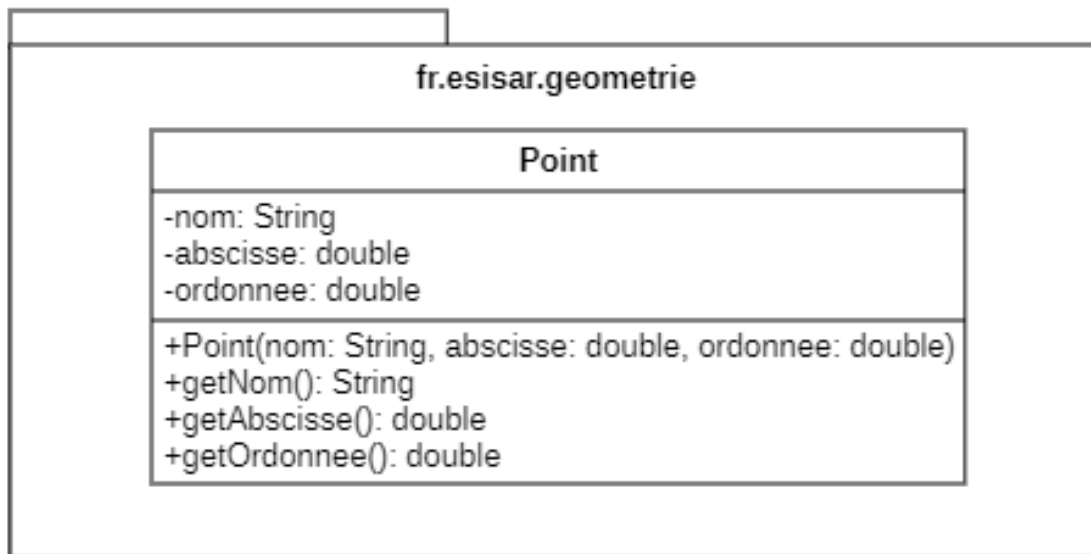


Diagramme de classe

Classe GeometrieTest

Créez, dans le package `fr.esisar.geometrie`, une classe `GeometrieTest`, qui contient une méthode `main()`.

Le corps de la méthode `main()` doit comprendre les instructions qui permettent de réaliser les opérations suivantes :

1. déclarer une variable nommée `pointA`, qui contiendra une référence vers un `=Point=` ;
2. initialiser cette variable grâce au constructeur avec les informations de la figure 12.³

¹ Le diagramme de classes est un outil de modélisation qui permet de représenter les classes et leurs relations. Ce diagramme utilise une notation normée définie en UML (Unified Modeling Language).

² A titre exceptionnel et dans un but pédagogique, ce diagramme de classes comprend la définition du constructeur et des getters. Dans un diagramme de classes représentant du code Java, les constructeurs, les getters/setters, les méthodes `hashCode()`, `equals()` et `toString()` ne sont jamais représentés.

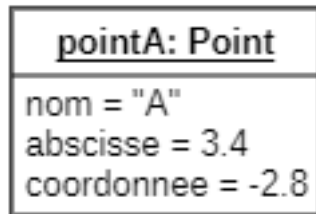


Diagramme d'objet

3. afficher les valeurs des attributs dans la console sous la forme :
nom(abscisse, ordonnée)

Les classes de test, s'appuient classiquement sur une utilisation intensive de l'instruction `assert`, comme vu lors du dernier cours de IN360. N'hésitez pas à vous en servir.

Questions :

1. Qu'est-ce qui est affiché si vous ajoutez, dans la méthode **main()** à la suite de votre code, la ligne suivante :

```
System.out.println(pointA);
```

Est-ce que cette valeur change lors d'exécutions différentes ?

A quoi cela correspond-il ?

2. Dans la définition de la classe `Point`, il n'a pas été défini de constructeur vide.

A votre avis, pourquoi ?

Quelles conséquences peut-il y avoir à définir un tel constructeur ?

```
public Point() {  
    super();  
}
```

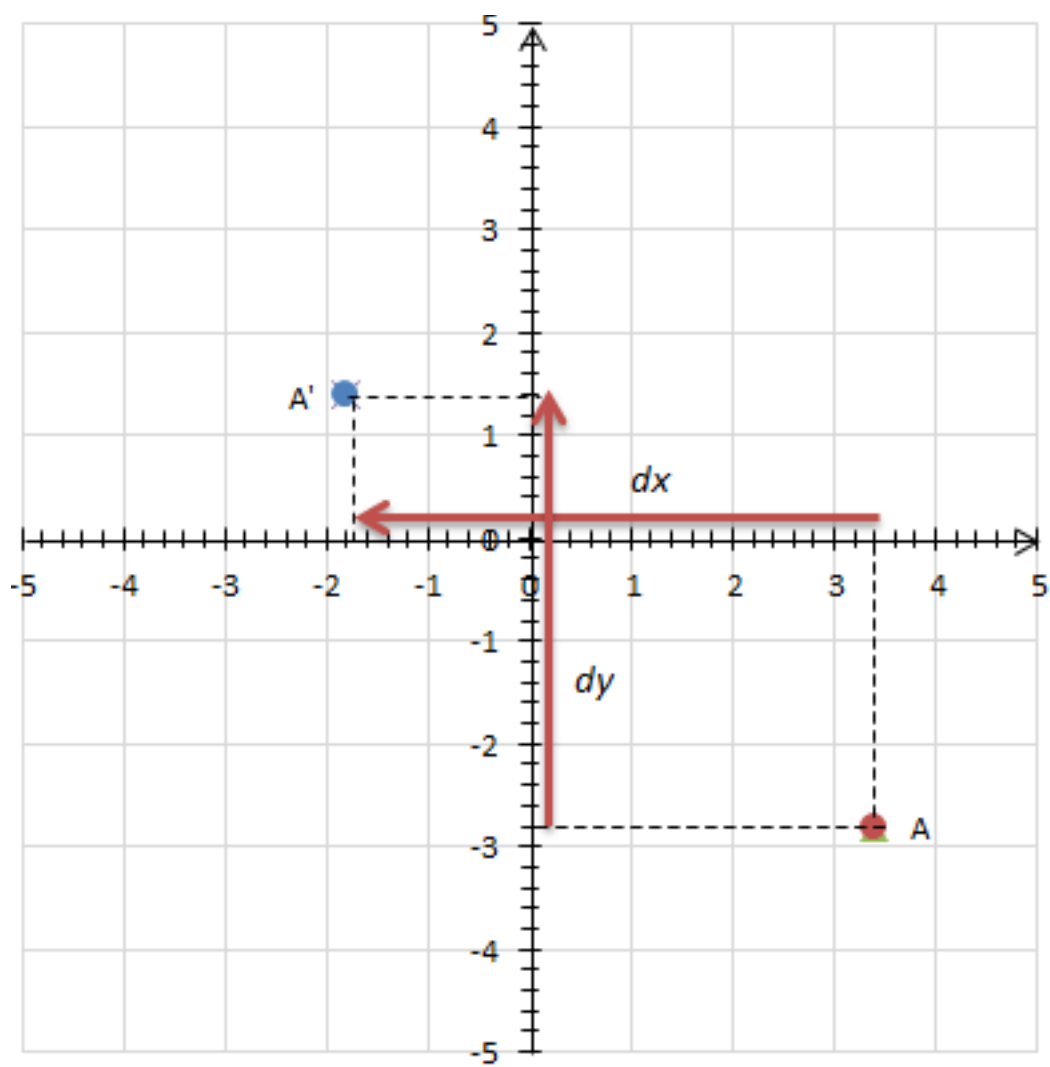
Ajout de méthodes à la classe `Point`

À la classe `Point`, ajoutez les deux méthodes suivantes (cf. figure 13) :

³ Le diagramme d'objets est aussi une notation UML, qui permet de représenter des objets (instances) et leurs relations. Le diagramme d'objets est utilisé pour montrer l'état des instances d'objet avant et après une interaction par exemple.

1. `deplacer(double dx, double dy)`, qui permet de faire un déplacement du point de la distance dx (respectivement dy) selon l'axe des abscisses (respectivement des ordonnées).
2. `tourner(double theta)`, qui permet de faire une rotation du point d'un angle θ (mesuré en degré) par rapport au centre du repère orthonormé.⁴

⁴ Vous utiliserez les constantes et les méthodes de la bibliothèque mathématiques de Java : `Math.PI`, `Math.cos()` et `Math.sin()`.



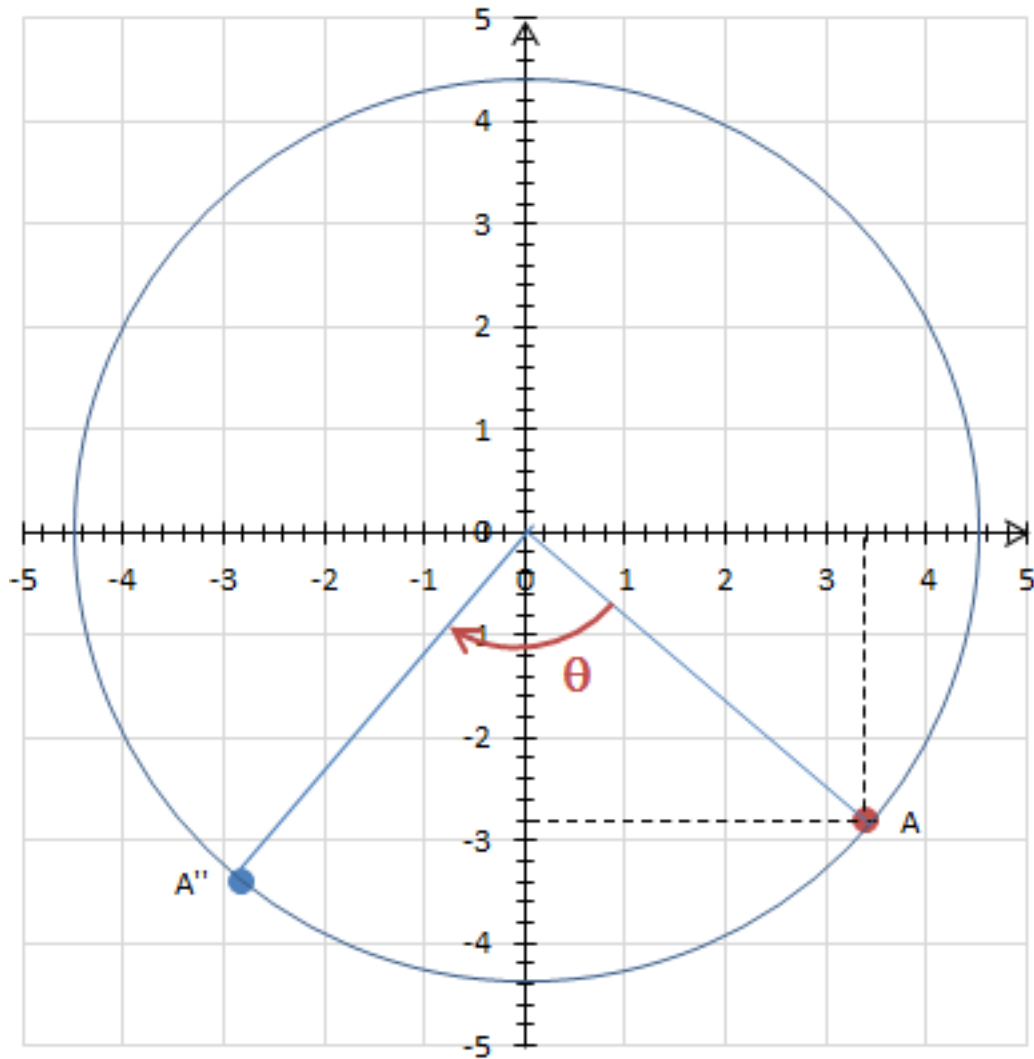


Diagramme d'objets

Donnez, dans un premier diagramme d'objets, la représentation du point A suite à un déplacement $dx = -5,2$ et $dy = 4,2$, puis, dans un deuxième diagramme d'objets, la représentation de ce point A, à partir de cette nouvelle position, après une rotation de -90° .

Code de la méthode `main()`

Complétez le code de la méthode `main()` pour réaliser les deux opérations précédentes et afficher les valeurs des coordonnées du point A.

Questions

1. Dans les méthodes `deplacer` et `tourner` de la classe `Point`, vous accédez aux valeurs des attributs avec :

- abscisse et ordonnee,
- ou bien this.abscisse et this.ordonnee,
- ou bien getAbscisse() et getOrdonnee(),
- ou bien encore this.getAbscisse() et =this.getOrdonnee()= ?

Quelle solution est la « meilleure » ?

2. A quoi sert le mot-clé =this= ?

Quand est-il utile ?

Classe Cercle

Un cercle est défini par un centre et un rayon. Le centre du cercle est un point du repère orthonormé.

Package `fr.esisar.geometrie`

Créez une classe Cercle, dans le package `fr.esisar.geometrie`, qui correspond à la définition donnée précédemment ainsi qu'au diagramme de classes de la figure 15.

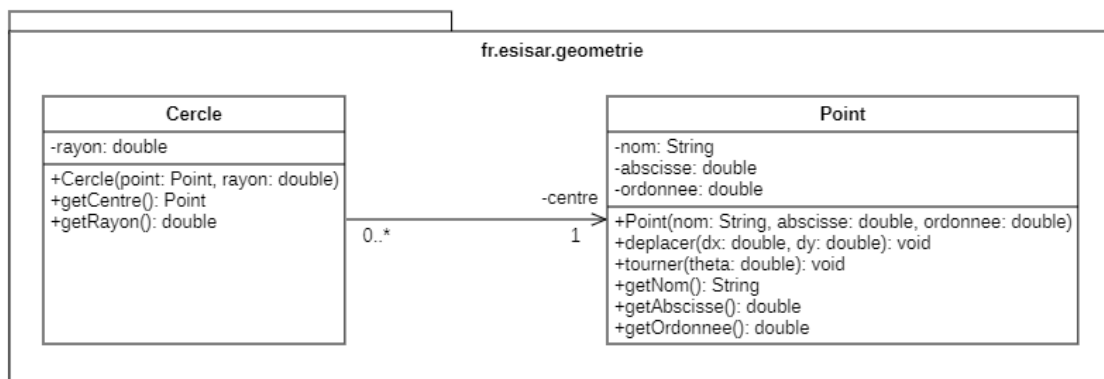


Diagramme de classe - Cercle et Point

Code de la méthode main()

Complétez le code de la méthode `main()` de la classe `GeometrieTest` pour créer un cercle de centre *M* de coordonnées $(4, 2; -1, 4)$ et de rayon 3, puis afficher ses caractéristiques. Cf. figure 16.

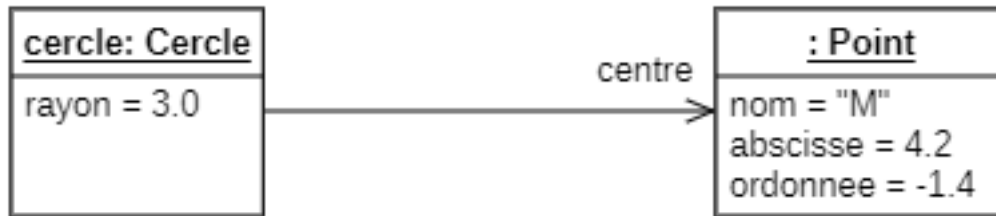


Diagramme d'objets - Cercle et Point

Ajout de deux méthodes

Complétez la classe Cercle pour ajouter deux méthodes :

- l'une `perimetre()`, qui calcule le périmètre du cercle,
- et l'autre `aire()`, qui calcule l'aire du cercle.

Testez ces deux méthodes dans la méthode `main()` de la classe `GeometrieTest`.

Questions

Lors de la création d'une instance de la classe `Cercle`, vaut-il mieux :

1. Créer une instance de point, puis passer cette instance au constructeur du cercle :

```
Point point = new Point("M", 4.2, -1.4);  
Cercle cercle = new Cercle(point, 3.0);
```

2. Créer une instance du cercle en passant au constructeur une instance « anonyme » d'un point :

```
Cercle cercle = new Cercle(new Point("M", 4.2, -1.4), 3.0);
```

Formater l'affichage des chaînes de caractères

Étant donnée l'utilisation du type `double` pour définir un ensemble d'éléments, dont notamment les coordonnées des points, vous avez obtenu des affichages tels que :

```
A(-1.8000000000000003,1.4000000000000004)
```

Dans la classe `Point`, ajoutez une méthode `void afficherPoint()`, qui affichera les coordonnées du point avec 2 chiffres après la virgule. Cette méthode aura un effet uniquement sur l'affichage et affichera :

```
A(-1.80,1.40)
```

Pour formater l’affichage, vous devez utiliser la fonction format de la classe String⁵ : `String.format(String, Object ...args): String`⁶ qui prend ses arguments comme la fonction `printf`⁷ en C.

Exemple :

```
String formattedStr =String.format("my numbers : %.4f", 3.14159f);
System.out.println(formattedStr);
```

Cependant, le format des nombres dépend des conventions des systèmes de numérotation qui, par exemple, diffèrent entre les systèmes anglo-saxons des autres systèmes pour la notation du séparateur de décimal. En notation anglo-saxonne, on utilise un point (e.g., 3.45) alors que pour les autres systèmes, on utilise une virgule (e.g., 3,45).

Par défaut en Java, une configuration par défaut est définie pour le système. Vous pouvez afficher le pays de référence avec : `Locale.getDefault().getCountry()`. Il est possible de le changer avec le code suivant pour passer ici en système anglo-saxon :

```
Locale locale = new Locale("en", "UK");
String.format(locale, "my numbers : %f", 15.21f);
```

Nota : Il semble que le constructeur `Locale("en", "UK")` soit obsolète depuis la version 19. Cela dit, il fonctionne encore correctement. On peut donc encore l'utiliser.

Au choix, configurez l’affichage de la méthode `afficher()` pour :

- soit afficher au format « français » les coordonnées : $A(-1,80;1,40)$ avec des virgules comme séparateurs de décimal et un point-virgule entre l’abscisse et l’ordonnée ;
- soit afficher au format « anglais » les coordonnées : $A(-1.80,1.40)$ avec des points comme séparateurs de décimal et une virgule entre l’abscisse et l’ordonnée.

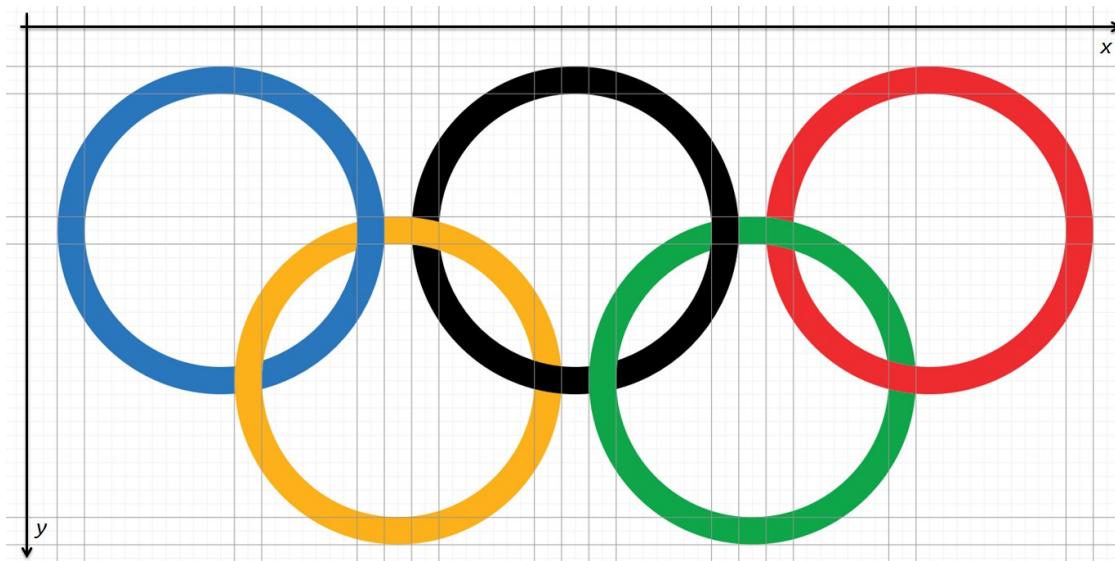
⁵ <https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#format-java.lang.String-java.lang.Object...>

⁶ <https://docs.oracle.com/javase/8/docs/technotes/guides/language/varargs.html>

⁷ <https://man7.org/linux/man-pages/man3/printf.3.html>

Produire un rendu graphique

L'objectif de cet exercice est de générer un fichier au format SVG⁸ qui représentera les anneaux olympiques. Les anneaux olympiques sont construits sur une grille géométrique stricte. Le diamètre d'un anneau est égal à 12 fois l'épaisseur d'un anneau. L'espace entre deux anneaux est égal à l'épaisseur d'un anneau. Cf. figure 17.



Anneaux olympiques

Un fichier svg contient une balise d'en-tête `<svg>` et une balise de fin `</svg>`. Entre ces balises, il est possible d'ajouter des commandes permettant de tracer des formes géométriques.

Exemple :

```
<svg
viewBox='0 0 100 100' style='background: white'
version='1.1' xmlns='http://www.w3.org/2000/svg'>

  <circle cx="15" cy="15" r="4" stroke="green" stroke-width="2" fill="yellow" />
</svg>
```

Vous trouverez plus d'informations sur les options des commandes SVG par exemple sur le site de W3schools :

https://www.w3schools.com/graphics/svg_intro.asp

⁸ https://fr.wikipedia.org/wiki/Scalable_Vector_Graphics

Compléter la classe Cercle pour réaliser un affichage spécifique pour la commande du tracé du cercle au format SVG. Le nom du centre peut être utilisé pour passer en paramètre la couleur. Si vous souhaitez aller plus loin, vous pouvez intégrer la couleur comme attribut de la classe Cercle. Dans un premier temps, les anneaux olympiques pourront ne pas être entrelacés.

Commentez le code de la méthode main() puis ajoutez les commandes, qui permettent d'afficher à l'écran l'ensemble du fichier SVG.

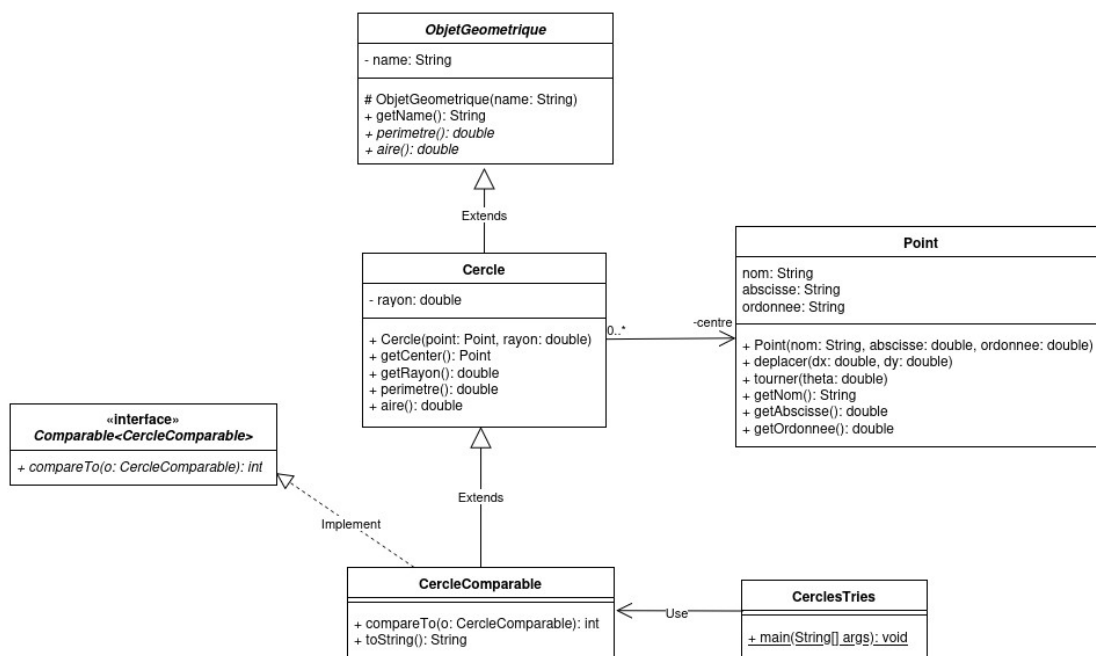
Exécutez en ligne de commandes votre programme et redirigez la sortie de l'exécution dans un fichier anneaux.svg. Vous pouvez afficher le contenu de ce fichier dans n'importe quel navigateur.

Et si vous entrelaciez maintenant les anneaux olympiques ?

Classe abstraite et interface

Dans une **seconde version de ce projet**, il est demandé de rendre votre code plus général, dans la perspective d'une utilisation avec d'autres objets géométriques.

L'arbre d'héritage est donné dans le diagramme classe de la figure 18 :



Arbre d'héritage

Classe abstraite `ObjetGeometrique`

Faire de la classe abstraite `ObjetGeometrique` générale donc la classe `Cercle` va hériter. On pourrait en effet imaginer manipuler d'autres classes telles que Carré, Rectangle, Losange, Triangle, etc. Nota, il n'est pas demandé de faire réaliser ces classes.

Cette classe `ObjetGeometrique` est abstraite car à la fois elle partage un certain nombre de propriétés entre tous les types d'objets géométriques mais également des méthodes abstraites telles que `aire` et `perimetre` dont le calcul diffère selon le type d'objet concerné.

Révision du code de la classe `Cercle`

Il vous est demandé de réviser le code initial de votre classe `Cercle` pour qu'il hérite de la classe `ObjetGeometrique`.

N'oubliez pas de définir les méthodes `aire` et `perimetre`.

Classe de test `TestObjetGeometrique`

Réaliser une classe de test qui, outre la fonction `main(String[] args)` définit les deux méthodes de test :

- `public static boolean surfaceIdentique(ObjetGeometrique objet1, ObjetGeometrique objet2)`
- `public static void afficheObjetGeometrique(ObjetGeometrique objet)` qui affiche sous forme textuelle les caractéristiques de l'objet en question.

Classe `CercleComparable`

Réaliser une classe `CercleComparable` qui hérite de `Cercle` et implémente l'interface `Comparable`.

Au delà du constructeur, cette classe devra définir les deux méthodes habituelles :

- `public int compareTo(CercleComparable o)`
- `public String toString()`

Classe `CerclesTriés`

Réaliser une classe `CerclesTriés` qui contient une méthode `main` dont le but est de trier une liste d'objets de type `CercleComparable`.

Gestion des exceptions

Classe CercleAvecException

Réaliser une classe CercleAvecException qui reprend le contenu de la classe Cercle que vous venez de développer, et qui gère une définition inappropriée de rayon. En effet, un rayon ne peut pas être négatif.

Pour cela, on fera appel à la notion d'*exception*.

Classe TestCercleAvecException

Bien sûr, en regard de cette nouvelle classe, vous développerez une classe de test TestCercleAvecException qui montrera comment se comporte votre programme lorsqu'on essaie de créer un Cercle avec un rayon négatif.

Développement d'une interface graphique avec JavaFX

Pour aller plus loin, vous pouvez développer une interface graphique vous permettant d'interagir avec les cercles créés : utilisation de la souris pour interactivement déplacer, agrandir, etc., vos cercles.

Pour cela, on utilisera la suite de développement graphique JavaFX. JavaFX est le nouvel outil standard à Java pour développer des interfaces graphiques multi-plateformes que ce soit sur ordinateurs ou sur tablettes et téléphone mobiles.

Nota : Ne pas utiliser l'outil de développement graphique SWING qui n'est plus maintenu.

Installation de JavaFX

Pour installer, compiler et exécuter JavaFX, utilisez la documentation générale de JavaFX

- <https://openjfx.io/openjfx-docs/#install-javafx>

Nota : Pensez bien à installer la même version que le JDK que vous utilisez.

Avant de vous lancer dans l'écriture de votre propre programme, il est hautement conseillé de testez votre installation avec l'exemple "HelloFX.java qui est donné dans la partie "Run HelloWorld using JavaFX SDK".

1. Pour les utilisateurs de machine Windows

Suivez pas à pas les instructions données pour Windows.

Exemple de commandes Windows :

pour stocker le chemin : `env:PATH_TO_FIX="C:\Users\makil\eclipse-workspace\javafx-sdk-24.0.1\lib"`

compilation : `javac --module-path $env:PATH_TO_FIX --add-modules javafx.controls
.\fr\esisar\projetfx\AfficherCercle.java`

exécution : `java --module-path $env:PATH_TO_FIX --add-modules javafx.controls
fr.esisar.projetfx.AfficherCercle`

2. Pour utiliser JavaFX dans Eclipse :

Documentation accessible ici :

https://media.pearsoncmg.com/ph/esm/ecs_liang_ijp_12/cw/content/supplements/EclipseJavaFX11.pdf

Classe AfficheCercle

Écrivez une classe `AfficheCercle` qui affiche un cercle au centre d'un cadre JavaFX ("*pane*").

Cette classe aura l'allure suivante

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class AfficheCercle extends Application {
    @Override // Réécrit la méthode start de la classe Application
    public void start(Stage primaryStage) {
        // Création d'un cercle and mise à jour de ses propriétés
        ....
        // Création d'un "pane" contenant le cercles
        ....
        // Création d'une "scene" et placement sur la "stage"
        ....
        primaryStage.show(); // Display the stage
    }
}
```


Interaction graphique avec les cercles

En tant qu'utilisateur, vous pouvez interagir avec l'interface graphique par le moyen de

- clic sur un bouton de l'interface graphique,
- déplacement de souris,
- clic souris,
- touche de clavier.

Cela génère un "événement" que Java capture à l'aide d'un "*handler*" (qui gère l'événement en question).

1. Sans utilisation de "handler"

Dans une première version, vous pouvez créer une interface graphique avec deux boutons qui vous permettent de

- a. élargir le cercle qui se trouve au centre de la fenêtre
- b. rétrécir le cercle.

2. Avec utilisation de "handler"

Dans une seconde version, créez une interface graphique qui capture les événements de la souris (de type "drag and drop") en faisant appel aux "EventHandler".

On reproduira les comportements précédents

- élargir un cercle,
- rétrécir un cercle.

Création interactive des anneaux olympiques

Vous pouvez vous fixer comme objectif ultime, de permettre à un utilisateur de créer les anneaux olympiques à l'aide de clics souris et de "drag and drop" pour

1. les positionner au bon endroit,
2. les agrandir à la demande,
3. les changer de couleur,
4. etc.